Lu Sun, and many more.

A Notebook on Artificial Intelligence

To my family, friends and communities members who have been dedicating to the presentation of this notebook, and to all students, researchers and faculty members who might find this notebook helpful.

Contents

Foreword **vii**

Preface **ix**

List of Figures **xi**

List of Tables **xiii**

I Introduction **1**

II Searching **3**

III Reasoning and Planning **5**

IV Basic Neural Networks **7**

1 Regression **9**

2 Perceptron **11**

3 Multi-layer Perceptron **13**

V Computer Vision **15**

4 Convolutional Neural Network **17**
4.1 Brief Review of CNN 17

VI Sequential Data Processing **19**

5 Recurrent Neural Network **21**
5.1 Brief Review of RNN 21

VII Large Language Model **25**

6 Transformer **27**
6.1 Limitations of CNN and RNN 27
6.1.1 A Brief Review of CNN and RNN 27
6.1.2 Attention Mechanism 28
6.2 Transformer 29

7 Large Language Model (Part 1: Theory) 31
7.1 Introduction to LLM 32
7.1.1 Existing Technologies and Gaps 32
7.1.2 Emergent Abilities 33
7.1.3 Milestone Techniques 34
7.1.4 Scaling Law 35
7.2 Data Corpus Resources and Existing LLMs 36
7.2.1 Data Corpus 37
7.2.2 Libraries and Packages 37
7.2.3 OpenAI Family 37
7.2.4 LLaMA Family 39
7.3 LLM Training Pipeline 44
7.3.1 Architecture Design 44
7.3.2 Training Set Preparation 44
7.3.3 Pre-training 44
7.3.4 Fine-Tuning 45
7.3.5 Model Evaluation 45
7.4 Multimodal LLM 45
7.5 Examples 45
7.5.1 LLM with Fine-Tuning 45
7.5.2 Semantic Search 45
7.5.3 Database Powered LLM 45
7.5.4 Multimodal LLM for Signal Processing 45
7.5.5 Multimodal LLM for Image Processing 45

8 Large Language Model (Part 2: Practice) 47
8.1 Quick Local Deployment 47
8.1.1 LLM Deployment with Ollama 47
8.1.2 LLM Application Integration with Langflow 48
8.2 Python Setup for LLM 48
8.2.1 Environment Creation and Packages Installation . . . 48
8.2.2 OpenAI API Powered LLM 50
8.2.3 Open-Source Model Powered LLM 51
8.3 Commonly Seen Frontier LLM Models 52

Bibliography 55

Foreword

If software and e-books can be made completely open-source, why not a notebook?

This brings me back to the summer of 2009 when I started my third year as a high school student in Harbin No. 3 High School. In the end of August when the results of Gaokao (National College Entrance Examination of China, annually held in July) are released, people from photocopy shops would start selling notebooks photocopies that they claim to be from the top scorers of the exam. Much curious as I was about what these notebooks look like, never have I expected myself to actually learn anything from them, mainly for the following three reasons.

First of all, some (in fact many) of these notebooks were more difficult to understand than the textbooks. I guess we cannot blame the top scorers for being so smart that they sometimes make things extremely brief or overwhelmingly complicated.

Secondly, why would I want to adapt to notebooks of others when I had my own notebooks which in my opinion should be just as good as theirs.

And lastly, as a student in the top-tier high school myself, I knew that the top scorers are probably my schoolmates or even friends. Why would I pay money to a complete stranger in a photocopy shop for my friends' notebook, rather than requesting a copy from them directly?

However, my mind changed after becoming an undergraduate student in 2010. There were so many modules and materials to learn for a college student, and as an unfortunate result, students were often distracted from digging deeply into a module (For those who were still able to do so, you have my highest respect). The situation became worse when I started pursuing my Ph.D. in 2014. As I had to focus on specific research areas entirely, I could hardly split much time on other irrelevant but still important and interesting contents.

In order to make a difference, I started enforcing myself reading articles beyond my comfort zone, which ended up motivating me to take notes to consolidate the knowledge. I used to work with hand-written notebooks. My very first notebook was on *Numerical Analysis*, an entrance level module for engineering background graduate students. Till today I still have dozens of these notebooks on my bookshelf. Eventually, it came to me: why not digitalizing them and make them accessible online and open-source, and let everyone read and edit it?

—

As most of the open-source software, this notebook does not come with any "warranty" of any kind, meaning that there is no guarantee that everything in this notebook is correct, and it is not peer reviewed. **Do NOT cite this notebook in your academic research paper or book!** If you find anything helpful here with your research, please trace back to the origin of the knowledge and confirm by yourself.

This notebook is suitable as:

- a quick reference guide;
- a brief introduction for beginners of an area;
- a "cheat sheet" for students to prepare for the exam or for lecturers to prepare the teaching materials.

This notebook is NOT suitable as:

- a direct research reference;
- a replacement of the textbook;

because as explained the notebook is NOT peer reviewed and after all, it is more of a notebook than a book. It is meant to be easy to read, not to be comprehensive.

Although this notebook is open-source, the reference materials of this notebook, including textbooks, journal papers, conference proceedings, etc., may not be open-source. Very likely many of these reference materials are licensed or copyrighted. Please legitimately access these materials and properly use them, should you decided to trace the origin of the knowledge.

Some of the figures in this notebook are drawn using Excalidraw, a very interesting tool for machine to emulate hand-writing. The Excalidraw project can be found in GitHub, *excalidraw/excalidraw*. Other figures may come from MATLAB, R, Python, and other computation engines. Should you decided to replicate the results in this notebook, make sure that you have legitimate access to these computation engines.

Preface

Artificial Intelligence (AI) was included as part of the control system notebook, as in early ages it was mostly used as a system identification tool in control systems.

With the advent of graphical processing units (GPU) in the 1990th and the Industry 4.0 initiatives in 2000th, deep learning network with massive training data has become possible, which significantly boost the performance of the artificial neural network (ANN)-based AI systems. Nowadays, AI has been growing rapidly with more and more successful demonstrations of useful cases such as computer vision and natural language processing.

Seeing that trend, AI relevant contents have been split from the control system notebook, and put here.

Special thanks go to the following listed materials, all of which have been very useful when drafting this notebook.

- Stuart J. Russell and Peter Norvig, *Artificial Intelligence A Modern Approach (3rd Edition)*
- Ed Donner, Ligency Team, *LLM Engineering: Master AI, Large Language Models & Agents*, Udemy, 2025

List of Figures

4.1 A demonstration of CNN kernel. The input is given by the white box (3D), and the kernel by the red box. 18

5.1 A example of RNN. 22

7.1 A cat with superpower. This picture is generated by DALL·E 3. 39
7.2 LLaMA family tree. 40
7.3 Alpaca fine-tuning pipeline. 42

8.1 An example of running Ollama with LLaMA 3.2. 48

List of Tables

7.1 Existing LLM models. 36
7.2 LLaMA models. 41

Part I

Introduction

Part II

Searching

Part III

Reasoning and Planning

Part IV

Basic Neural Networks

1

Regression

CONTENTS

2

Perceptron

CONTENTS

3

Multi-layer Perceptron

CONTENTS

Part V

Computer Vision

4

Convolutional Neural Network

CONTENTS

4.1 Brief Review of CNN 17

“nobreak

4.1 Brief Review of CNN

CNN is a type of ANN structure designed to handle grid-like data. It is effective when dealing when data spatially correlated, thus becomes very popular in computer vision. It defines “kernel” that aggregates nearby pixels information before sending it to a dense network.

A CNN kernel, also known as a filter, is a small matrix of weights that slides across the input image or feature map to perform a mathematical operation called convolution. An demonstration of CNN kernel is given in Fig. 4.1. Multiple kernels can be defined on the same layer to handle the same feature map, each kernel associated with an output channel. In practice, each kernel or channel is designed to detect a specific features in the feature map. For example, there might be a kernel detecting edges, while a second kernel detect color codes.

Notice that CNN differs quite largely from transformer in the problems they are expected to address. CNN is more for spatial data processing such as image processing, while transformer targets more on sequential data processing such as natural language processing and machine translation.

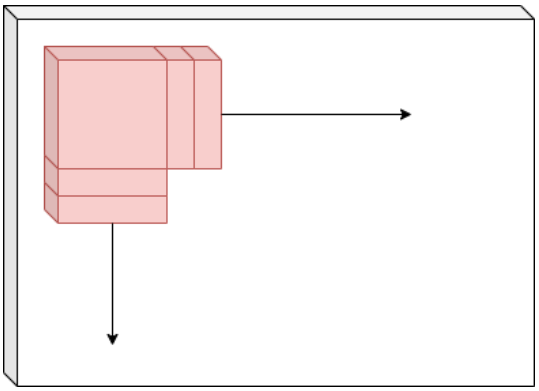

FIGURE 4.1
A demonstration of CNN kernel. The input is given by the white box (3D), and the kernel by the red box.

Part VI

Sequential Data Processing

5

Recurrent Neural Network

CONTENTS

5.1 Brief Review of RNN 21

"nobreak

5.1 Brief Review of RNN

RNN is a connectionist model with the ability to selectively pass information across sequence steps [6]. It is good at handling sequence of data such as voice message, text contents, or a flow of images (videos). It is worth mentioning that the "sequence" does not necessarily mean a time sequence. Nevertheless, without losing generality, we will consider time sequence in the review for simplicity and convenience.

Denote inputs $x(1), x(2), ..., x(k), ...$ where $x(k)$ is a vector sampled at time instant k. The length of the sequence may be finite or infinite. In the case of finite sequence, its maximum sample index is denoted by T. For example, in the context of natural language processing, each input might be a word in a dictionary. For example, $x(1)$ = "Pandas", $x(2)$ = "are", $x(3)$ = "so", $x(4)$ = "cute", $x(5)$ = "!". The corresponding target output sequence is given by $y(1), y(2), ..., y(k), ...$, respectively.

RNN differs from the conventional dense ANN by introducing "recurrent edges", which allows the output of hidden layers at $k-1$ be used as additional inputs to the system at k. This means, at any time k, the input of the system includes both $x(k)$ and also selected $h(k-1)$, where $h(\cdot)$ is the outputs of hidden layers. We can think of the "weights" of a trained RNN the "long-term memory" that does not change with specific sequence of inputs, while the information passing through recurrent edges the "short-term memory" that links previous inputs with future inputs.

A demonstration is given in Fig. 5.1. Notice that each hidden layer is a multi-input-multi-output subsystem containing multiple nodes. Different from a conventional dense ANN, each hidden layer takes additional inputs from its corresponding hidden layer in the previous instant. It is also common to see

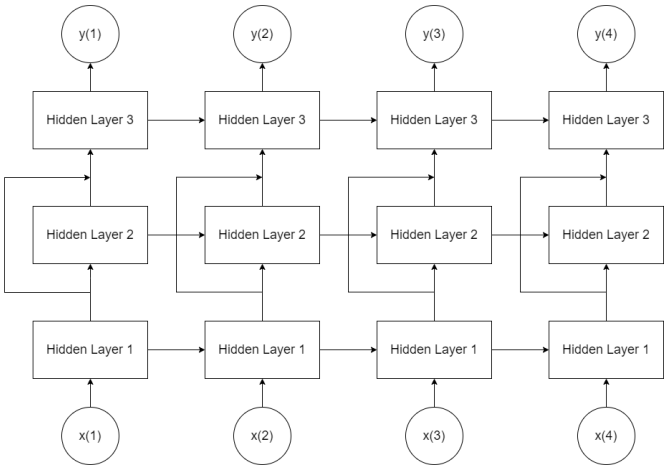

FIGURE 5.1
A example of RNN.

"bypass" (this is widely used in different ANN structures, not unique to RNN) for better performance of the system.

RNN has some limitations. One of the major problem is that it is difficult to train an RNN even for the basic standard feedforward networks. The optimization of RNN is NP-complete. It is especially difficult for RNN to learn long-range dependencies due to the vanishing and exploding gradients problem that could occur when backpropagating errors across many timestamps (long sequence) [6]. This is one of the main challenges why RNN has difficulties building on long-range dependencies. The vanishing and exploding gradient problems are caused by the structure of the system as well as the backpropagation-based training methods.

Different approaches have been proposed to prevent vanishing and exploding gradient problems. Famous ones among these approaches include strategical weight initialization, long short term memory (LSTM), gated recurrent units (GRUs), skip connections, and more. Many of these approaches try to reduce the effect of vanishing and exploding gradient problems by carefully design the ANN structures. For example, both LSTM and GRUs introduce "memory cells" with built-in "gates" that balance and control the flow of information from previous cells versus current inputs. The cells are used to replace the traditional perceptron nodes. These approaches have made the training of RNN a feasible problem. LSTM and GRUs are almost certainly used in modern RNNs.

Bidirectional RNN (BRNN) is proposed at about the same time with LSTM. It allows information to travel not only from previous hidden layers to future layers, but also from future hidden layers to previous layers. LSTM and BRNN can be used together to boost the RNN performance. Notice that

BRNN cannot run continuously as it requires fixed endpoints in both the future and the past. It is useful for prediction over a sequence of fixed length, such as part-of-speech tagging in natural language processing.

Another problem that people have found during the training of RNN is local optima. However, recent studies have shown that local optima is not as serious issue as we might thought when the network is large, since many critical points are actually saddle points rather than local minima.

Successful implementations of the above RNN structures include natural language translation such as [9] where a encoder-decoder structure is used, each is a LSTM. Another example is image captioning, where the AI tries to explain what is in an image using texts. A solution to this is to use CNN to encode the image, and use LSTM to decode to generate texts. Following similar ideas is hand-writing reorganization.

Part VII

Large Language Model

6

Transformer

CONTENTS

6.1 Limitations of CNN and RNN 27
6.1.1 A Brief Review of CNN and RNN 27
6.1.2 Attention Mechanism 28
6.2 Transformer 29

This chapter introduces the development of "transformer", an AI framework initially proposed in 2017 in a paper "Attention is All You Need" [11]. It has magically revolutionized how we train AI for NLP. The "encoder-decoder" framework it uses has inspired the development of generative AI.

6.1 Limitations of CNN and RNN

CNN and RNN have been introduced previously. This section does not focus on the detailed introduction of CNN and RNN mechanisms. Instead, only their main features are reviewed, and their limitations discussed. This section does not reflect the state-of-art of researches in CNN and RNN.

6.1.1 A Brief Review of CNN and RNN

The conventional feedforward network with dense ANN is widely used. It can be trained systematically using back-propagation methods such as stochastic gradient descent. However, dense ANN is not very good at interpreting correlations among the inputs. At least, it is not good at doing it efficiently.

For example, consider the input to be pixels of an image of size $3 \times 128 \times 128$, where 3 corresponds to the RGB of the image, and 128×128 its pixel size. It is obvious to a human that the pixel at $(1, 1)$ should definitely has a closer relationship with the one at $(1, 2)$ (they are probably forming an object in the image together) than $(128, 128)$. The same applies to sequential inputs. Consider a signal sampled continuously. The first and second samples are very likely to be more strongly correlated than the first and last samples.

In a conventional dense ANN, the correlation cannot be captured efficiently. In contrast, a dense ANN would treat all the inputs "equally" in a symmetric manner. To enforce the ANN to "address" the correlation, it would require a lot more layers and nodes (which is often considered low-efficient), and requires more data points during the training. Other problems of conventional dense ANN include, for example, the lack of ability in handling data with arbitrary length.

CNN and RNN try to tackle the above problems by implementing a "preprocessing" stage, where the correlation of the spacial and sequential data is first abstracted using some mechanism, and the correlation information is sent as (additional) inputs to the followed dense ANN.

6.1.2 Attention Mechanism

"Attention" was not a new concept invented in [11]. It was proposed a few years earlier than the transformer, and had been integrated into several CNN and RNN models to boost their performances to capture long-range dependencies. In these researches, attention mechanism is more often a supporting role than the main character. It is quite surprising indeed that when getting rid of CNN and RNN and purely relying on attention mechanism, everything gets better: more efficient in training and better in performance.

Does this mean that the RNN, which we had been relying on so much, is in fact the drawback to NLP? The following is the answer of ChatGPT-4 of this question.

Q: By getting rid of RNN and using attention mechanism alone, natural language processing performance gets better. Is this correct? Does this mean that RNN is a drawback to natural language processing?

A: Yes, by getting rid of RNNs and using the attention mechanism alone, the Transformer model demonstrated improved performance in natural language processing tasks. However, it's important to note that RNNs are not inherently a drawback to NLP; they were successfully used for many tasks before the advent of Transformers.

RNNs, particularly LSTM (Long Short-Term Memory) and GRU (Gated Recurrent Unit) variants, were designed to handle sequences and capture dependencies in the data. They have been effective in many NLP tasks, such as language modeling, machine translation, and sentiment analysis. However, RNNs do have certain limitations, such as difficulty in capturing long-range dependencies and sequential processing, which can lead to slower training and inference times.

The attention mechanism, as employed in the Transformer model, addresses some of these limitations. It enables parallel processing of input sequences, leading to faster training and inference, and effectively cap-

tures long-range dependencies without the vanishing gradient problem that can affect RNNs.

While Transformers have shown superior performance in many NLP tasks, RNNs still have their place in certain scenarios, especially when working with resource-constrained environments or smaller datasets. They can be more lightweight and computationally efficient compared to large-scale Transformer models.

In summary, RNNs are not a drawback to NLP per se, but their limitations have been mitigated by the introduction of the attention mechanism in Transformer models, which has led to improved performance in a wide range of NLP tasks.

(April 14, 2023, ChatGPT-4)

6.2 Transformer

While CNN is mainly used for spatial data processing such as computer vision, both RNN and transformer are mainly used for sequential data processing. It is worth mention in the very beginning that transformer does not guarantee superior performance in all scenarios comparing with traditional RNN-based natural language processing models such as LSTM.

For one thing, transformer consumes larger computational capabilities. Transformer processes data in batch, while RNN does them in sequence, meaning that RNN can response faster in some real-time applications. It is difficult for the transformer to process super-long text due to the computational burden (quadratic computational complexity with respect to sequence length), while RNN can process arbitrarily long text, although the later suffers from capturing long-range dependencies.

With the above been said, the transformer does demonstrated superior performance than RNN in many tasks. The mechanism and the reasons why it performs better in these occasions are introduced as follows.

7

Large Language Model (Part 1: Theory)

CONTENTS

7.1 Introduction to LLM 32
7.1.1 Existing Technologies and Gaps 32
7.1.2 Emergent Abilities 33
7.1.3 Milestone Techniques 34
7.1.4 Scaling Law 35
7.2 Data Corpus Resources and Existing LLMs 36
7.2.1 Data Corpus 36
7.2.2 Libraries and Packages 37
7.2.3 OpenAI Family 37
7.2.4 LLaMA Family 39
7.3 LLM Training Pipeline 44
7.3.1 Architecture Design 44
7.3.2 Training Set Preparation 44
7.3.3 Pre-training 44
7.3.4 Fine-Tuning 44
7.3.5 Model Evaluation 45
7.4 Multimodal LLM 45
7.5 Examples 45
7.5.1 LLM with Fine-Tuning 45
7.5.2 Semantic Search 45
7.5.3 Database Powered LLM 45
7.5.4 Multimodal LLM for Signal Processing 45
7.5.5 Multimodal LLM for Image Processing 45

This chapter serves as an overview of **large language model** (LLM), a popular and successful NLP practice. LLM can not only process human languages but also generate new contents based on human language written requirements. The introduction of multimodal LLM further allows LLM to understand and generate not just writings but also pictures and videos. The theory and technology behind LLM are briefly discussed.

Nowadays, it is possible for a regular user to leverage on commercialized models to develop his own applications. Given a powerful machine, a user can even deploy LLMs locally in a few steps. Open-source LLMs are available so

that the user does not need to train a model from scratch. The deployment of LLMs and the interaction with them are introduced in consecutive chapters.

7.1 Introduction to LLM

This section reviews existing NLP methods and their limitations, and discusses the gap LLM bridges.

7.1.1 Existing Technologies and Gaps

There have been variety of ways to model natural languages. For example, consider the following sentence:

"I am thirsty. Please give me a bottle of _____."

It is quite natural that a human would likely to put "water" or "tea" in the blank. This is obvious because human has a dictionary of words that he can choose in his mind, and he has been reinforced of learning "a bottle of water" expression in many occasions. In addition, it makes sense to a human that when someone is thirsty, he would look for water.

The challenge using AI on the above task is to build the "dictionary" in the machine and quantitatively analyze what word or phrase would make the most sense to be filled into the blank. Many models have been proposed to solve this problem, some of which have been evolving over time in the past decades since 1990th.

Statistical Language Models

In the early days when AI and ANN were not popular, **statistical language models** (SLM) have been the most popular tool to model natural languages. SLM assumes that a sentence is a Markov process, and the last word depends on the context created by the most recent n words. SLM with a fixed context length of n is also called an n-gram language model.

Conventionally, the paring information from n words to the next word is obtained from data corpus and stored in a table-like structure. When running the model, it looks up the table for the most probable next word based on the earlier n words. Smoothing technologies are used to handle zeros, i.e., when the record is not found in the table.

An obvious issue of SLM is that the computation and storage of the model increase exponentially with the size of n. This limits the context information that the model can use for prediction, hence setting a low performance ceiling. Not to mention that even with a large data corpus, zeros can still happen and the model performance is always an issue in such occasions.

Neural Language Models

With the introduction of ANN, in particular RNN, **Neural Language Models** (NLM) became popular. RNN-based NLM builds the word prediction function conditioned on the aggregated context features abstracted and passed recurrently from current and previous input sequence.

RNN is not a perfect one-stop solution either. The training of RNN can be difficult due to vanishing and exploding gradient problems. This limits the depth that RNN can go. When handling long sequence of words, the performance of RNN drops significantly because it is weak at building long-term dependencies.

Pre-trained Language Models and Large Language Models

As introduced in details in earlier chapters, attention mechanism has been proposed to tackle the long-term dependency problem of RNN. Transformer architecture, which relies purely on encoder, decoder and attention mechanism without using RNN is then proposed. It has been verified that transformer architecture is good at abstracting information from sequential data, in particular, natural languages. With a transformer, it becomes possible to build very deep neural networks and have it trained efficiently with big-size data corpus.

Language models based on different transformer-based architectures are often called **Pre-trained Language Models** (PLM). LLM can be taken as a subset of PLM. The main difference between an LLM and a regular PLM is the size of the model. The scaling of the model from hundreds of millions of parameters for PLMs to tens or hundreds of billions of parameters for LLMs introduces emergent abilities such as in-context learning to the model, significantly enhancing its capability and intelligence. As of this writing, it is not very clear how the these abilities suddenly emerge with the size of the model.

7.1.2 Emergent Abilities

When the size of the model becomes large, usually to the order of at least a few billions parameters, they suddenly gain emergent abilities. Details are discussed as follows.

In-context Learning

In-Context Learning (ICL) allows the behavior of the model to be manipulated via not training or fine-tuning of the parameters, but via instructions and demonstrations given as part of the input.

ICL plays an important part in LLM implementation, as it is the basis of prompt engineering. When the LLM is large, it is possible to use prompt engineering instead of fine-tuning to complete a task following user defined

instructions. This reduces the training cost and makes the implementation more flexible.

Instruction Following

Supervised learning is commonly used in training and fine-tuning a model. In the training set, we need to provide the model "examples". An example includes the input, some good responses, and some bad responses.

When comes to LLM, it is possible to fine-tune a model for a specific task without using the aforementioned examples. Instead, just give it step-by-step instructions. LLM is able to perform well with these tasks described only by instructions. This is known as instruction tuning.

It is worth mentioning that instruction following is also possible in ICL. Give the LLM instructions in prompt engineering without examples, and the LLM is likely to be able to finish the tasks.

Step-By-Step Reasoning

When a model is asked to complete a complicated task that involves multiple steps, it may fail to accomplish the task. With a well fine-tuned LLM, the model might be able to break the task into multiple sub-tasks via chain-of-thought (CoT) prompting strategy, and solve them step-by-step till the final result is obtained.

It has been observed that an LLM with at least $100B$ parameters is likely to have good step-by-step reasoning abilities.

7.1.3 Milestone Techniques

This section looks back into the progression tree of LLM, and lists down milestone techniques that make LLM what it is today. It is the breakthrough in these areas that revolutionizes LLM development.

Big Data

The performance of LLM relies on both the modal size and the training data size. It is the advent in internet, Web 3.0, Industry 4.0, IoT and cloud computing/storage that makes collection and aggregation of big data possible.

Almost every large-size enterprise, both IT companies and conventional industrial companies, have their internal databases. The database can be used to train domain-knowledge LLM. Nowadays, there are many open data sources of community LLMs. Such examples include BookCorpus, CommonCrawl, Reddit posts (with high upvotes), Wikipedia, and many more. These open-source datasets make training LLM for community projects possible.

Large Model and Efficient Training

The invention of CPU-based neural networks and transformer architecture making creating and training large scale LLM possible. Both closed and

open-source LLMs have been proposed, including GPT series by OpenAI and LLaMA family by Meta AI and the community.

Manly libraries have been released to the public to help building, training and fine-tuning LLMs, such as `transformers`, a Python library for building transformer models. Many such libraries are tying up with PyTorch and TensorFlow to provide LLM related functions.

More about these models and libraries are introduced in later sections.

Fine-Tuning

Technologies such as LoRA has made fine-tuning easier than before.

Prompt Engineering

There have been a lot of practices on how to make LLM flexible and more efficient in solving particular tasks via prompt engineering.

LLM on Edge Devices

Many efforts have been put into edge-device based LLMs. The target is to develop LLM that consumes less memory, storage and computation while not sacrificing a lot of performance.

API and Interface

Multi-modal LLM has enabled different types of inputs to the LLM, not limited to natural language but also sequential signals and even pictures.

Many tools and software have developed APIs for LLM. These tools enhance computation and online information retrieval capabilities of LLM.

7.1.4 Scaling Law

The performance of an LLM, usually referring to its capability in accurately and correctly complete a task, is affect by many factors such as the model architecture, model size, training data set size and quality, etc. Though it is clear that with the scaling up of the system the performance is usually improved, there is no analytical expression that gives full insights about how these factors affect the performance quantitatively.

A **scaling law** tries to quantitatively describe the performance of an LLM as a function of model size and other factors. Many scaling laws have been proposed, many of which obtained from empirical experiments and they may work only within a given range of model size.

Just as an example, OpenAI proposed KM scaling law in 2020 that describes LLM cross entropy loss as a function of model size, training data set

TABLE 7.1
Existing LLM models.

Availability	Model Name	Size	Training	Release Time
open	CodeGen	16	$577B$	2022-Mar
	LLaMA	65	$1.4T$	2023-Feb
	CodeGen2	16	$400B$	2023-May
	LLaMA2	70	$2T$	2023-Jul
closed	GPT-3	175	$300B$	2020-May
	Codex*	12	$100B$	2021-Jul
	GPT-4	—	—	2023-Mar

Model size is given in number of parameters in billion. Training data size is given in number of tokens.
* Codex is trained on top of GPT-3.

size and training computation as follows.

$$
\begin{aligned}
L(N) &= \left(\frac{N_c}{N}\right)^{\alpha_N} \\
L(D) &= \left(\frac{D_c}{D}\right)^{\alpha_D} \\
L(C) &= \left(\frac{C_c}{C}\right)^{\alpha_C}
\end{aligned}
$$

where N, D and C denote the model size, dataset size and training computation, respectively. The rests are constants whose value can be obtained via calibration.

This scaling law works for models with $22M$ to $23B$ parameters. It is assumed that the analysis of a factor can be done independently without other parameters being a bottleneck.

7.2 Data Corpus Resources and Existing LLMs

Data corpus is available online for LLM training. Many libraries such as PyTorch compatible Python packages have been developed to automate the training procedures and to convenient the users. Many LLMs, both open-source and closed-source, are available on the market. A summary is given in Table 7.1. Notice that the table only covers a small portion of existing models in the market.

The existing resources and solutions are briefly introduced in this section.

7.2.1 Data Corpus

Enterprise and individual users may have their own closed source data corpus for training and fine-tuning. In this section, only open source data corpora are discussed.

7.2.2 Libraries and Packages

"nobreak

7.2.3 OpenAI Family

OpenAI started investigating language models before the proposition of transformer. In its early days, RNN was explored as the most promising model for natural language. In 2017 when the transformer model was proposed, OpenAI quickly adapted their language model to this new architecture, and as a result generative pre-training (GPT) series has been proposed.

GPT-1

GPT-1, OpenAI's first transformer based PLM was proposed in 2018. GPT-1 has $117M$ parameters in the model and it adopts a decoder-only architecture, which is different from the original transformer proposal which has a encoder-decoder architecture. GPT-1 was trained via a two-stage procedure, the first stage unsupervised pre-training and the second stage supervised fine-tuning. This two-stage training pipeline, or something of the similar kind, has been adopted by many LLMs coming after.

GPT-2

GPT-2 is an improvement of GPT-1. It uses much larger number of parameters of $1.5B$ in the model, and it was trained on a much larger dataset WebText. With larger model and training data size, GPT-2 is targeted to be a multi-task solver. The model can be formulated by the following probabilistic form

$$\text{Pre-trained LLM} \quad \equiv \quad P\left(\text{output}|\text{input}, \text{task}\right)$$

In the above formulation, each NLP task can be considered as the word prediction problem based on a subset of the word next, and can be trained during the unsupervised learning stage. Unsupervised pre-training has since then become the most important stage for the LLM to gain knowledge for general tasks.

GPT-3, GPT-3.5 and Chat-GPT

It is clear now that GPT-2 has $1.5B$ parameters which is too few for an LLM to gain emergent abilities. It is GPT-3 with $175B$ parameters trained

on $300B$ tokens that made a capability leap and bring LLM to everyone's attention.

It is GPT-3 that for the first time introduces emergent abilities such as ICL. GPT-3 not only accomplishes commonly seen tasks to test LLM capabilities with flying color, but also demonstrates features not shown by other models before, such as reasoning and domain adaption.

OpenAI has developed many task-oriented models that use GPT-3 as the base model. For example, for coding, Codex was introduced. Codex is basically GPT-3 fine-tuned using code database such as GitHub. Comparing with GPT-3, Codex is able to reason and solve complex mathematical problems, and realize them in codes. RL had already been used to fine-tune and improve performance for GPT-2. The same has been applied on GPT-3. Furthermore, reinforcement learning with human feedback (RLHF) is introduced for GPT-3 that allows the model to continue learning from human demonstrations.

With the above enhancements, i.e. code-based fine-tuning, RL and RLHF, GPT-3.5 has been developed. GPT-3.5 is an enhanced version of GPT-3 and it is obtained from GPT-3 via transfer learning. GPT-3.5 is also scaled up in size, with a parameter number of $335B$.

Chat-GPT was developed on top of GPT-3.5 (and later another version was released developed on top of with GPT-4). It is GPT-3.5 fine-tuned specially optimized for dialogue. It demonstrates how LLM can be used in everyone's daily life as an assistant, consultant and a copilot. Chat-GPT demonstrated good performance in communicating of humans than other GPT models. This is because Chat-GPT is trained on human-generated conversations of both human and AI roles. Later on, plug-ins and APIs are enabled on Chat-GPT, to allow it to communicate with other software.

GPT-4

GPT-4 is a multimodal LLM, meaning that it can not only take in natural languages, but also other signals such as images. Comparing with GPT-3.5, GPT-4 demonstrates further enhanced capabilities. It is claimed that GPT-4 is safer to use and is less likely to generate toxic and misleading information.

Just as a quick demonstration of its multimodal feature, pass the following Fig. 7.1 (notice that this figure itself is generated by an AI model) to GPT-4 with instruction "What is in this picture?". GPT-4 is able to response the following:

"The image showcases a majestic cat standing in a fantastical landscape. The cat emits a luminous, ethereal glow, and its fur and tail appear to be infused with swirling, fiery or cosmic energies in brilliant hues of red and gold. These radiant swirls surround the cat, adding to its supernatural aura. Behind the cat, there's a dramatic night sky with clouds illuminated by a hidden light source, possibly lightning or a celestial event. In the foreground, the terrain appears to be a combination of grassy knolls and fiery, ember-lit ground, fur-

ther emphasizing the magical and otherworldly nature of the scene. The entire setting evokes a sense of wonder and fantasy."

FIGURE 7.1
A cat with superpower. This picture is generated by DALL·E 3.

Other OpenAI Models

DALL·E is the model OpenAI uses to generate images. As of this writing, its latest version, DALL·E 3, has been integrated with the latest ChatGPT. Figure 7.1 gives an example where DALL·E 3 is used to generate a image of a superpower cat.

OpenAI Codex, as introduced earlier, is the GPT model optimized for code generation. As of this writing, Codex has been deprecated because its capability has been integrated into ChatGPT. It is possible to ask ChatGPT to generate a piece of code following user's instruction and description.

OpenAI Codex is the engine that powers GitHub copilot. In this sense, OpenAI Codex remains a live in a different format: not as a standalone software or model, but more as an API.

7.2.4 LLaMA Family

Large Language Model Meta AI (LLaMA) is the LLM model developed by Meta AI. Different from most of the AI models (GPT-3 and onward) developed by OpenAI, LLaMA is open-source hence has a wide availability. Many efforts in the community have made modifications and improvements to LLaMA, making a big family of models with different characteristics.

As of this writing, a family tree of LLaMA is shown in Figure 7.2. The picture is from [12]. The source of the picture is given in the GitHub repository of the paper. Only a small portion of models in the family tree is briefly

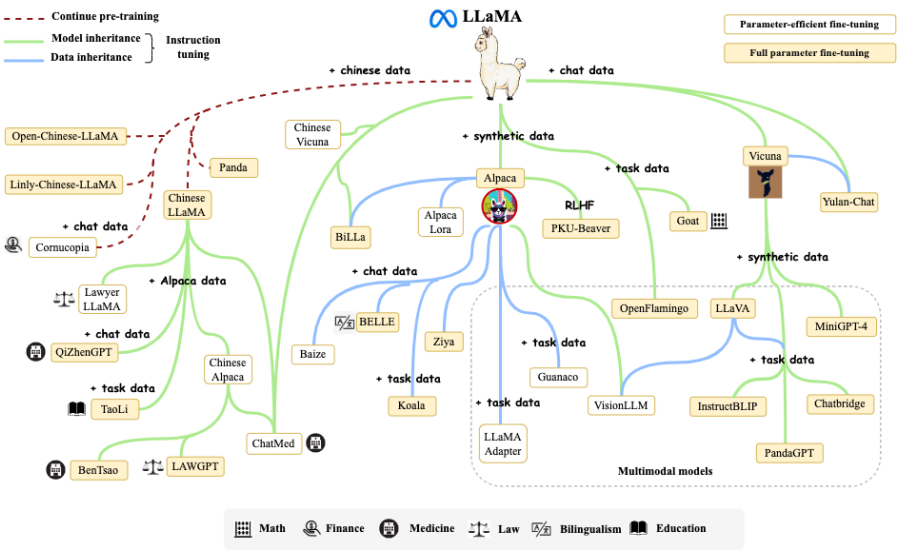

FIGURE 7.2
LLaMA family tree.

introduced here.

LLaMA

LLaMA, comparing with GPT-3 which was used as a benchmark, is smaller in model size (maximum $65B$ parameters VS $175B$ parameters in GPT-3) but larger and better in training data size and quality (maximum $1.4T$ tokens VS $300B$ tokens in GPT-3). As a result, LLaMA is able to achieve generally better performance than GPT-3 with less implementation cost due to the small size. LLaMA demonstrates that training data is equality important as model size. It is possible to reach the same level of performance with a small ($< 100B$ parameters) but well-trained model.

LLaMA's first release includes 4 models of different model and training sizes. Details are summarized in Table 7.2. The training dataset is purely open-source, including Common Crawl, C4 Dataset, GitHub, Wikipedia, public domain books, arXiv and Stack Exchange.

Technical wise, LLaMA has some innovations on top of the original transformer proposition [11] n the normalization method, activation function, and embedding methods. More details will be introduced in later sections.

Many open-source tools and packages have been developed to fine-tune LLaMA and its variations. More about fine-tuning, such as LoRA[5] and

TABLE 7.2
LLaMA models.

Name	Model Size	Training Dataset Size
LLaMA 7B	$6.7B$	$1T$
LLaMA 13B	$13.0B$	$1T$
LLaMA 33B	$32.5B$	$1.4T$
LLaMA 65B	$65.2B$	$1.4T$

Model size is given in number of parameters in billion. Training data size is given in number of tokens.

QLoRA[4], are introduced in more details in later sections. Notice that fine-tuning or re-training of a model often cost a lot of computational power and vector memory.

Stanford Alpaca

Stanford Alpaca 7B is Standford's practice in fine-tuning LLaMA 7B. The running cost of this fine-tuning is impressively small (hundreds of USD), and the resulted model has a performance comparable with GPT-3.5 which has $335B$ parameters. It is impressive to see that a small $7B$ model can compete with a large $335B$ model by careful training and fine-tuning. This might be partially because Alpaca 7B is fine-tuned from LLaMA 7B using knowledge distillation from GPT-3.5 (also known as text-davinci-003).

The details of training Stanford Alpaca can be found in [10]. A brief highlight is given below. The fine-tuning pipeline of Alpaca is shown in Fig. 7.3. It is a 2-stage process as follows.

1. Generating instruction-following examples using GPT-3.5 (text-davinci-003).

2. Fine-tune LLaMA 7B using the examples.

Examples of the generated instruction and response pairs are given below. The full list is available from the GitHub repository.

```
{
       "instruction": "What are the three primary colors?",
       "input": "",
       "output": "The three primary colors are red, blue, and yellow."
},
{
       "instruction": "Give three tips for staying healthy.",
       "input": "",
       "output": "1.Eat a balanced diet and make sure to include plenty
            of fruits and vegetables. \n2. Exercise regularly to keep
           your body active and strong. \n3. Get enough sleep and
           maintain a consistent sleep schedule."
```

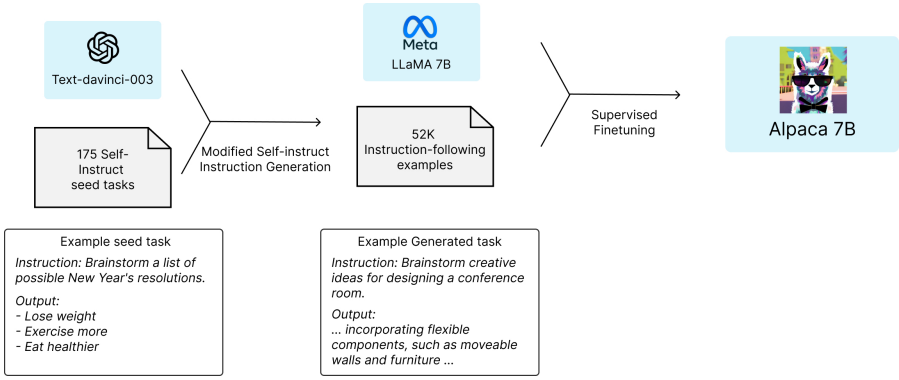

FIGURE 7.3
Alpaca fine-tuning pipeline.

```
},
{
        "instruction": "Extract the facts from the paragraph.",
        "input": "Online education continues to become more popular for
            schools and students alike. Its advantages are generally
            lower costs, less commitment and the ability to study at a
            time, place and pace that suits the student.",
        "output": "- Online education is becoming increasingly popular.\
            n- It has several advantages such as lower costs, less
            commitment and the ability to study at one\u2019s own time
            and pace."
},
{
        "instruction": "Arrange the given numbers in ascending order.",
        "input": "2, 4, 0, 8, 3",
        "output": "0, 2, 3, 4, 8"
}
```

The fine-tuning code of Alpaca is from Hugging Face. Python packages such as `numpy`, `torch`, `transformers`, etc., are required. Check `requirements.txt` and `train.py` in the repository for more details. Call `train.py` using something like the following

```
torchrun --nproc_per_node=4 --master_port=<your_random_port> train.py \
        --model_name_or_path <
            your_path_to_hf_converted_llama_ckpt_and_tokenizer> \
        --data_path ./alpaca_data.json \
        --bf16 True \
        --output_dir <your_output_dir> \
        --num_train_epochs 3 \
        --per_device_train_batch_size 4 \
        --per_device_eval_batch_size 4 \
```

```
    --gradient_accumulation_steps 8 \
    --evaluation_strategy "no" \
    --save_strategy "steps" \
    --save_steps 2000 \
    --save_total_limit 1 \
    --learning_rate 2e-5 \
    --weight_decay 0. \
    --warmup_ratio 0.03 \
    --lr_scheduler_type "cosine" \
    --logging_steps 1 \
    --fsdp "full_shard auto_wrap" \
    --fsdp_transformer_layer_cls_to_wrap 'LlamaDecoderLayer' \
    --tf32 True
```

where `./alpaca_data.json` is the JSON file that contains all the instruction and response pairs.

Notice that later on, Stanford Alpaca 13B is also developed based on LLaMA 13B.

Vicuna

Vicuna 13B is an open-source chatbot fine-tuned on top of LLaMA using $70K$ user-shared conversations collected from ShareGPT (a platform where users share their ChatGPT conversation histories). Its performance is comparable with GPT-3.5 and Bard, and it is slightly better than LLaMA 13B and Alpaca 13B in general.

See [3] for more details.

LLaVA and MiniGPT-4

Large Language and Vision Assistant (LLaVA) is a multimodal LLM that combines a vision encoder and Vicuna for general purpose visual and language understanding. More details are given in [8] and [7]. Similar with GPT-4, its multimodal encoders allows it take both natural language and image as inputs. An example is given below.

Pass the same Fig. 7.1 to LLaVA with the same instruction "What is in this picture?". LLaVA is able to give the following response:

"The image features a cat standing on a hill, with a fiery background and a cloudy sky. The cat appears to be looking at the viewer, and there is a sense of motion in the scene."

Similar with LLaVA, MiniGPT-4 is a also multimodal LLM that enhances vision interpretation. More details are given in [13]. It uses different techniques and realizations from LLaVA. Pass the same Fig. 7.1 to MiniGPT-4 with the same instruction "What is in this picture?". MiniGPT-4 is able to give the following response:

"The image shows a white cat with flames coming out of its fur, standing on

a hill at sunset. The cat's eyes are closed, and it appears to be in a meditative state. The background is a dark sky with clouds and a full moon. The overall mood of the image is peaceful and mystical."

Notice that both LLaVA and MiniGPT-4 model can be downloaded from their GitHub and HuggingFace repositories. As of this writing, live demos are also available for these models.

Multimodal LLM and "Any-to-Any" LLM are not the main forecast of this chapter. More details will be introduced in later chapters.

LLaMA 2

LLaMA 2 is the next generation open source LLM following the original LLaMA. The model is free for research and commercial use. LLaMA 2 70B has $70B$ parameters and it is pre-trained with $2T$ tokens of training data.

7.3 LLM Training Pipeline

"nobreak

7.3.1 Architecture Design

Encoder and Decoder

Normalization Method

Activation Function

Position Embedding Method

Attention Mechanism

7.3.2 Training Set Preparation

Commonly used data corpora include the following.

7.3.3 Pre-training

"nobreak

7.3.4 Fine-Tuning

“nobreak

7.3.5 Model Evaluation

“nobreak

7.4 Multimodal LLM

“nobreak

7.5 Examples

“nobreak

7.5.1 LLM with Fine-Tuning

“nobreak

7.5.2 Semantic Search

“nobreak

7.5.3 Database Powered LLM

“nobreak

7.5.4 Multimodal LLM for Signal Processing

“nobreak

7.5.5 Multimodal LLM for Image Processing

8

Large Language Model (Part 2: Practice)

CONTENTS

8.1 Quick Local Deployment 47
8.1.1 LLM Deployment with Ollama 47
8.1.2 LLM Application Integration with Langflow 48
8.2 Python Setup for LLM 48
8.2.1 Environment Creation and Packages Installation 48
8.2.2 OpenAI API Powered LLM 50
8.2.3 Open-Source Model Powered LLM 51
8.3 Commonly Seen Frontier LLM Models 52

This chapter studies the selection, deployment and use of LLMs in production environment for different tasks.

8.1 Quick Local Deployment

Ollama and Langflow are introduced. Ollama allows fast deployment of an open-source LLM on a local computer, while Langflow is a low-code tool that allows a developer to easily integrate LLMs, both local and commercially cloud-based, into an application.

8.1.1 LLM Deployment with Ollama

Nowadays it is convenient to quickly deploy an open-source LLM on a PC with necessary hardware. One of the simplest way of doing that is to leverage **Ollama**, an open-source tool that allows the deployment of commonly seen open-source LLMs such as LLaMA, DeepSeek, etc., on a local machine computer. During the installation of Ollama, it detects and configures the GPU of the machine and configure it accordingly, saving the user's time.

Once Ollama is installed, use

```
ollama run <model name>
```

to download and start a model. A screenshot is given in Fig. 8.1 as an example that runs LLaMA 3.2 with Ollama on a PC using the above command.

```
C:\Users\sunlu>ollama run llama3.2
>>> Tell me something about llama as in large language model
A Llama is a type of artificial intelligence (AI) designed to process and generate human-like language. It's a
part of the Meta AI library, which aims to build conversational AI models that can understand and respond to
natural language inputs.

Llama stands for "Large Language Model Meta AI," and it's a transformer-based architecture that uses
self-attention mechanisms to process and analyze large amounts of text data. This allows Llama to learn patterns
and relationships in language, enabling it to generate coherent and context-specific responses.

The key characteristics of Llama include:

1. **Natural Language Processing (NLP)**: Llama is designed to understand and process human language, allowing it
to generate responses that are similar to those produced by humans.
2. **Transformers**: Llama uses transformer architecture, which is a type of neural network specifically designed
for NLP tasks. This allows the model to process sequential data, such as text, efficiently.
3. **Self-attention mechanisms**: Llama employs self-attention mechanisms, which enable it to weigh the importance
of different parts of the input text and generate responses that take into account the context.
```

FIGURE 8.1
An example of running Ollama with LLaMA 3.2.

The deployed LLM does not come with any fancy graphical interface, but instead with the basic CLI. The user has the freedom to further deploy interfaces for applications on top of the basic interface.

A list of Ollama supported LLMs are given in [2]. As of this writing, famous ones include `deepseek-r1` (1.5B to 671B), `llama3.3` (70B), `llama3.2` (1B, 3B), `gemma3` (1B to 27B), and a lot more.

8.1.2 LLM Application Integration with Langflow

TBA

8.2 Python Setup for LLM

Most of today's LLM development and deployment are done using Python. In this section, a Python environment is setup for that. Anaconda, an open-source Python and R languages distribution, is used to setup the environment.

8.2.1 Environment Creation and Packages Installation

It is recommended to collect all the necessary libraries in a file, such as `environment.yml`, and use

```
conda env create -f environment.yml --name <environment name>
```

to create an environment and install the packages all together. Anaconda shall figure out the dependencies and compatibility of the packages, and have everything installed correctly.

An example of such YAML file is given below. This example is taken from [1].

```
channels:
  - conda-forge
  - defaults
dependencies:
  - python=3.11
  - pip
  - python-dotenv
  - requests
  - numpy
  - pandas
  - scipy
  - pytorch
  - jupyterlab
  - ipywidgets
  - matplotlib
  - scikit-learn
  - chromadb
  - jupyter-dash
  - sentencepiece
  - pyarrow
  - faiss-cpu
  - pip:
    - beautifulsoup4
    - plotly
    - bitsandbytes
    - transformers
    - sentence-transformers
    - datasets
    - accelerate
    - openai
    - anthropic
    - google-generativeai
    - gradio
    - gensim
    - modal
    - ollama
    - psutil
    - setuptools
    - speedtest-cli
    - langchain
    - langchain-core
    - langchain-text-splitters
    - langchain-openai
    - langchain-chroma
    - langchain-community
    - faiss-cpu
    - feedparser
```

```
  - twilio
  - pydub
```

Among the libraries shown above, some are commonly used across all Python and machine learning projects such as `numpy`, `pandas` and `scikit-learn`, while others are LLM specific packages such as `transformer`, `ollama` and `langchain`.

Environment Solutions of Anaconda Versus PyPA

Anaconda is a Python distribution developed and maintained by Anaconda Inc. It provides `conda`, a useful tool, to manage packages and environments. Anaconda provides reliable services to professional data scientists, cooperations as well as the free-of-charge services to the community. Python also has its native packages and environments management tools known as `pip` and `venv` developed and maintained by Python Packaging Authority (PyPA).

Both `conda` and `pip` allow users to create an environment and install packages from a file. The commands are

```
conda env create -f <filename> --name <environment name>
```

and

```
python -m venv <environment name>
pip install -r <filename>
```

respectively.

It is of the user's choice whether to use Anaconda (or its light version, Miniconda) or Pip to manage the packages. As long as this notebook goes, both of them should fulfill the needs. As of this writing, it is recommended that data scientists, researchers and lectures shall use Anaconda (non-commercial) and Miniconda (commercial), while software engineers, Pip.

8.2.2 OpenAI API Powered LLM

If the user has decided to use a commercialized LLM model via its API key, he needs to register an account with the LLM provider, such as OpenAI, and create a new API key, and added it to the project as an environmental variable. Notice that the use of the API key often introduces costs to be paid to the LLM provider. The details about the registration of the account and the creation of the API key are not included in this notebook.

This section gives a brief demo of how to connect to OpenAI using its API. The model to be used is `gpt-4o-mini`. It is assumed that API key has been created and stored under `.env` file in the project folder as follows.

```
OPENAI_API_KEY=sk-proj-<content>
```

The following code demonstrates the connection of the local Python plant to the OpenAI LLM model. The "completions" API is used, which asks the LLM to complete a conversation.

```
import os
from openai import OpenAI

api_key = os.getenv('OPENAI_API_KEY')

openai = OpenAI()

message = "Hello, GPT! I am connecting you via your API. Let's see how
    it works."
response = openai.chat.completions.create(model="gpt-4o-mini", messages
    =[{"role":"user", "content":message}])
print(response.choices[0].message.content)
```

A response similar to the following can be obtained.

```
Hello! It's great to hear that you're connecting via the API. How can I
     assist you today?
```

A typical `messages` that is passed to OpenAI often looks like the following

```
messages = [
   {"role": "system", "content": "<system prompt>"},
   {"role": "user", "content": "<user prompt>"}
]
```

where `<system prompt>` tells LLM the basic setup, such as what role the LLM shall play and what it should do, whereas `user prompt` gives the user specific data that the LLM needs to process.

8.2.3 Open-Source Model Powered LLM

In earlier sections it has been introduced that Ollama can be used to conveniently deploy an open-source LLM on the local machine, and it also provides a CLI where the user can chat with the LLM. It is possible to connect a Python program to that LLM using the interface that Ollama provides.

Make sure that the model is running in the backend. Start the model and check the model status using `ollma run` and `ollama ps` respectively.

The python program can connect to the model either via HTTP calls, or with Python `ollama` package. For the sake of convenience, only the latter is introduced.

```
import ollama

MODEL = "llama3.2"
messages = [
```

```
    {"role": "user", "content": "Describe some of the business
        applications of Generative AI"}
]

response = ollama.chat(model=MODEL, messages=messages)
print(response['message']['content'])
```

OpenAI's Python package also provides tools to connect to local LLMs like what has been deployed via Ollama.

```
from openai import OpenAI
ollama_via_openai = OpenAI(base_url='http://localhost:11434/v1',
    api_key='ollama')

response = ollama_via_openai.chat.completions.create(
    model=MODEL,
    messages=messages
)

print(response.choices[0].message.content)
```

8.3 Commonly Seen Frontier LLM Models

This section introduces commonly seen frontier LLM models as of this writing. Some of the most famous ones are listed below.

- Open-source
  - LLaMA, and its variations (Meta)
  - Gemma (Google)
  - Qwen (Alibaba Cloud)
  - Deepseek-R1 (Deepseek)
  - Phi (Microsoft)
- Closed-source
  - GPT (OpenAI)
  - Gemini (Google)
  - Claude (Anthropic)

There are a couple of ways to use these models. Examples are given below.

Use LLM with companies. Companies such as OpenAI, Google, Microsoft and Deepseek provide web-based chatbot interface where a user can directly

chat with the model. Many of these companies also provide APIs that allow user program to connect to models running on the cloud.

Use LLM with cloud service providers. Nowadays many cloud providers including AWS, Microsoft Azure, Google Cloud Service, etc., that allow the user to deploy LLM models on their servers. For example, AWS provides Amazon Bedrock which is its frontier model that can be easily deployed on AWS. Similar applies to other major cloud service providers.

Use LLM on premises. Users can train their own models, or download the weights of existing models and deploy them locally. Many open-source models are downloadable from websites such as Hugging Face. Tools such as Ollama can help with quickly deployment of these models locally.

Bibliography

[1] LLM engineering (ed donner). `https://github.com/ed-donner/llm_engineering`. Accessed: 2025-04-15.

[2] Ollama library. `https://ollama.com/library`. Accessed: 2025-04-14.

[3] Wei-Lin Chiang, Zhuohan Li, Zi Lin, Ying Sheng, Zhanghao Wu, Hao Zhang, Lianmin Zheng, Siyuan Zhuang, Yonghao Zhuang, Joseph E. Gonzalez, Ion Stoica, and Eric P. Xing. Vicuna: An open-source chatbot impressing gpt-4 with 90%* chatgpt quality, March 2023.

[4] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. Qlora: Efficient finetuning of quantized llms. *arXiv preprint arXiv:2305.14314*, 2023.

[5] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models, 2021.

[6] Zachary C. Lipton, John Berkowitz, and Charles Elkan. A critical review of recurrent neural networks for sequence learning, 2015.

[7] Haotian Liu, Chunyuan Li, Yuheng Li, and Yong Jae Lee. Improved baselines with visual instruction tuning, 2023.

[8] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual instruction tuning. In *NeurIPS*, 2023.

[9] Ilya Sutskever, Oriol Vinyals, and Quoc V Le. Sequence to sequence learning with neural networks. *Advances in neural information processing systems*, 27, 2014.

[10] Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. Stanford alpaca: An instruction-following llama model. `https://github.com/tatsu-lab/stanford_alpaca`, 2023.

[11] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need, 2017.

[12] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, et al. A survey of large language models. *arXiv preprint arXiv:2303.18223*, 2023.

[13] Deyao Zhu, Jun Chen, Xiaoqian Shen, Xiang Li, and Mohamed Elhoseiny. Minigpt-4: Enhancing vision-language understanding with advanced large language models. *arXiv preprint arXiv:2304.10592*, 2023.